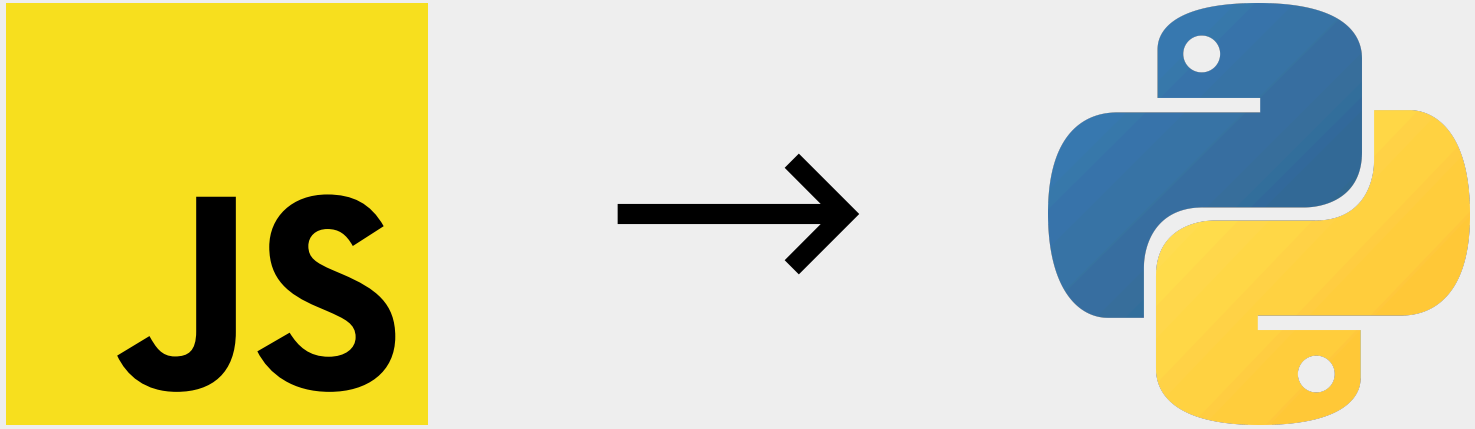
The Nature of Code, but Python



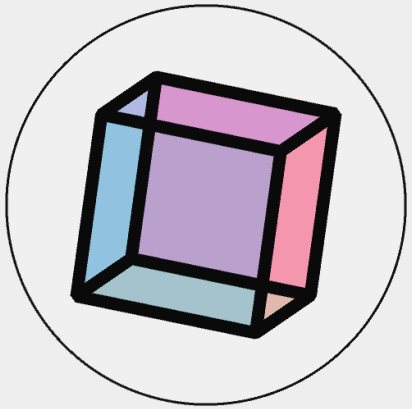
Introduction



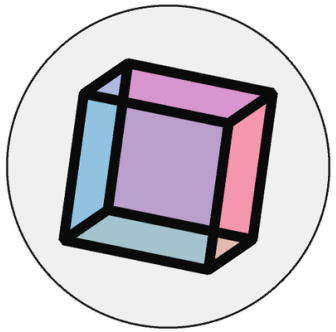
https://tisch.nyu.edu/itp/news/fall-2024/daniel-shiffman--itp-faculty-member--launches-new-book---the-nat


JS

p5.js →



https://py5coding.org



Search Ctrl + K

- Welcome to py5!

About py5

- What is py5?
- py5 Community
- How To Contribute
- py5 maintainer's blog

Getting Started

- Install py5
- Tips for Processing Java Users
- Special Notes for macOS Users
- The Five py5 Modes

Welcome to py5!

py5 is a new version of Processing for Python. The goal of py5 is to create a version of Processing that is integrated into the Python ecosystem. Built into the library are thoughtful choices about how to best get py5 to work with other popular Python libraries and tools such as Jupyter, numpy, shapely, trimesh, matplotlib, and Pillow.

py5 is an excellent choice for educators looking to teach Python in the context of creative coding and is currently used in classrooms all around the world. This website's documentation includes introductory tutorials as well as extensive reference documentation, complete with example code.

Basic Example

Here is a basic example of a py5 Sketch:

```
def setup():
    size(400, 400)
    rect_mode(CENTER)


def draw():
    square(mouse_x, mouse_y, 10)


def mouse_clicked():
    fill(random_int(255), random_int(255), random_int(255))


run_sketch()
```

https://py5coding.org

Introduction » py5 Features

- Modern Python: Full Python 3.x+ support
- Processing Engine: Fast Java graphics backend via JPype
- Java Bridge: Can load and use many existing Java Processing libraries
- Data Science Ready: Native integration with NumPy, Pandas
- Any Editor: Works in Jupyter Notebooks, VS Code, ...
- Fast Pixels: Vectorized pixel manipulation using NumPy arrays
- Flexible Modes: Choose Imported (beginner), Module, or Class

Introduction » Run py5

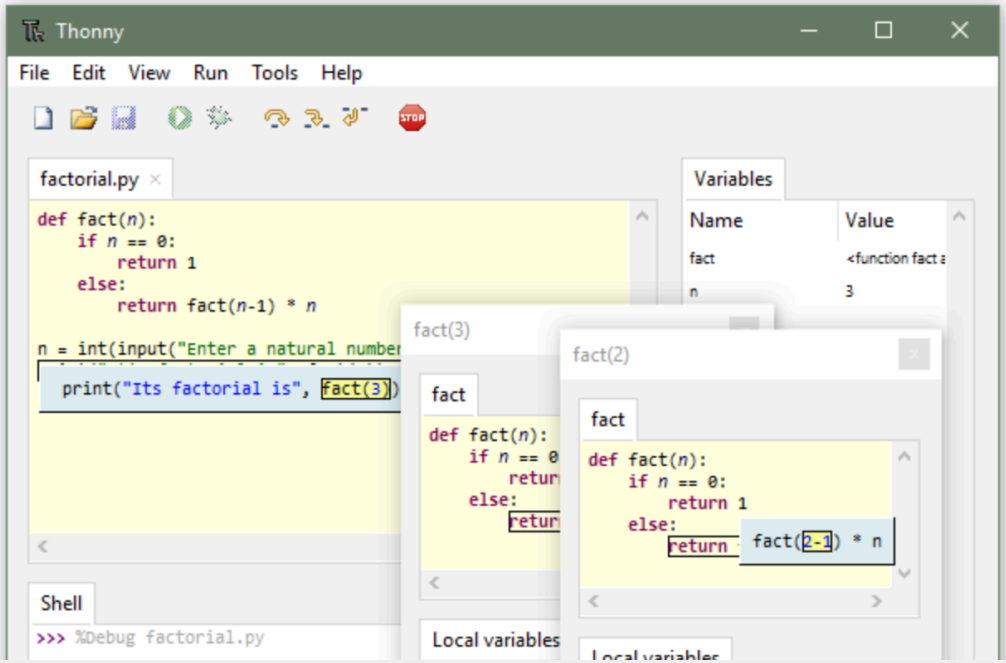
- Standard Python script: VS Code, PyCharm, terminal, …
- Jupyter Notebook: Via standard ipykernel, or py5 kernel
- Thonny-py5mode: Thonny IDE + py5 plugin ←

Thonny 5 is dedicated to Ukraine fighting the Russian invasion.
🇺🇦 Please support Ukraine! 🇺🇦

Thonny

Python IDE for beginners

Download version 5.0.0 for
Windows • Mac • Linux



https://thonny.org

Introduction » Thonny Features

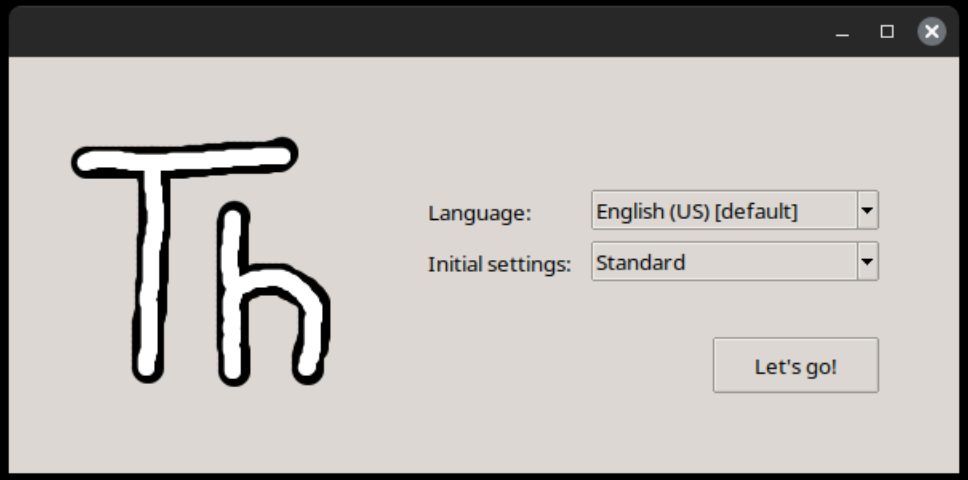
- Beginner-Focused: Minimal, uncluttered interface designed for learning
- Bundled Python: Includes a built-in Python 3 runtime
- Cross-Platform: Runs seamlessly on Linux, macOS, Windows
- Fully Portable: Runs directly from a USB drive without admin rights
- Several Neat Panels: Interactive shell, plotter, outline, and more

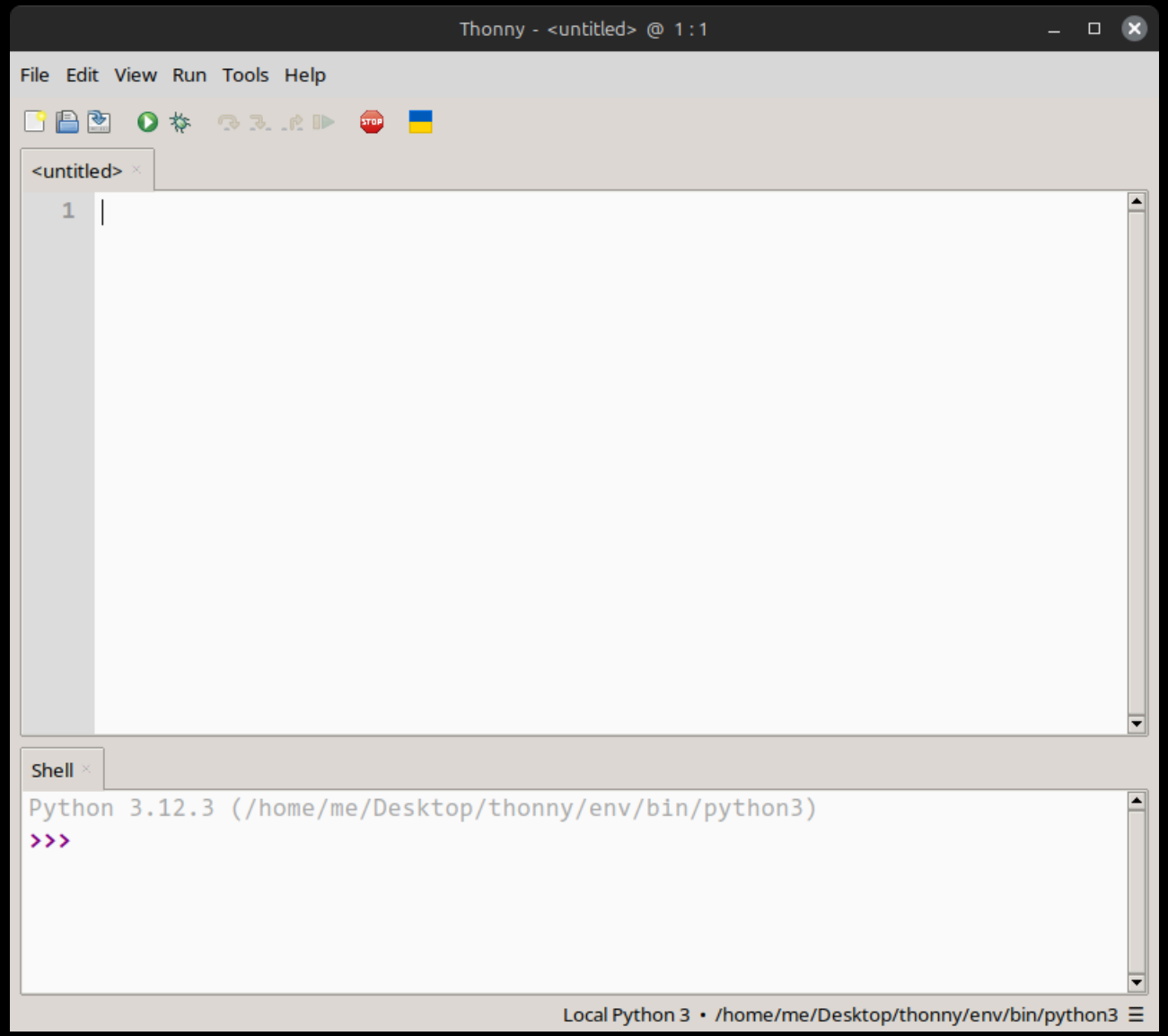
Introduction » Thonny-py5mode Installation

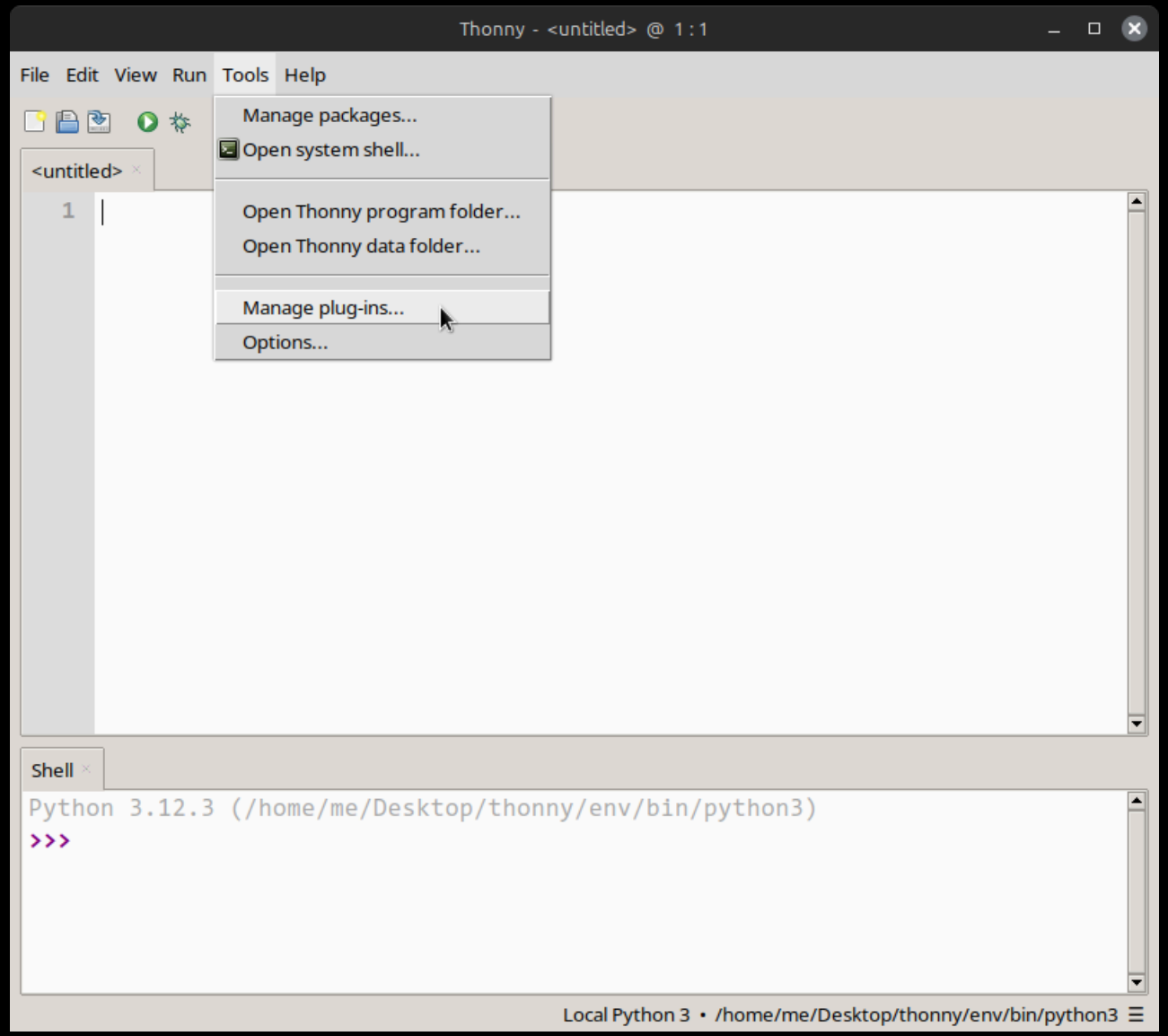
A plugin that turns the Thonny IDE into a Python 3 replacement for the original Processing IDE.

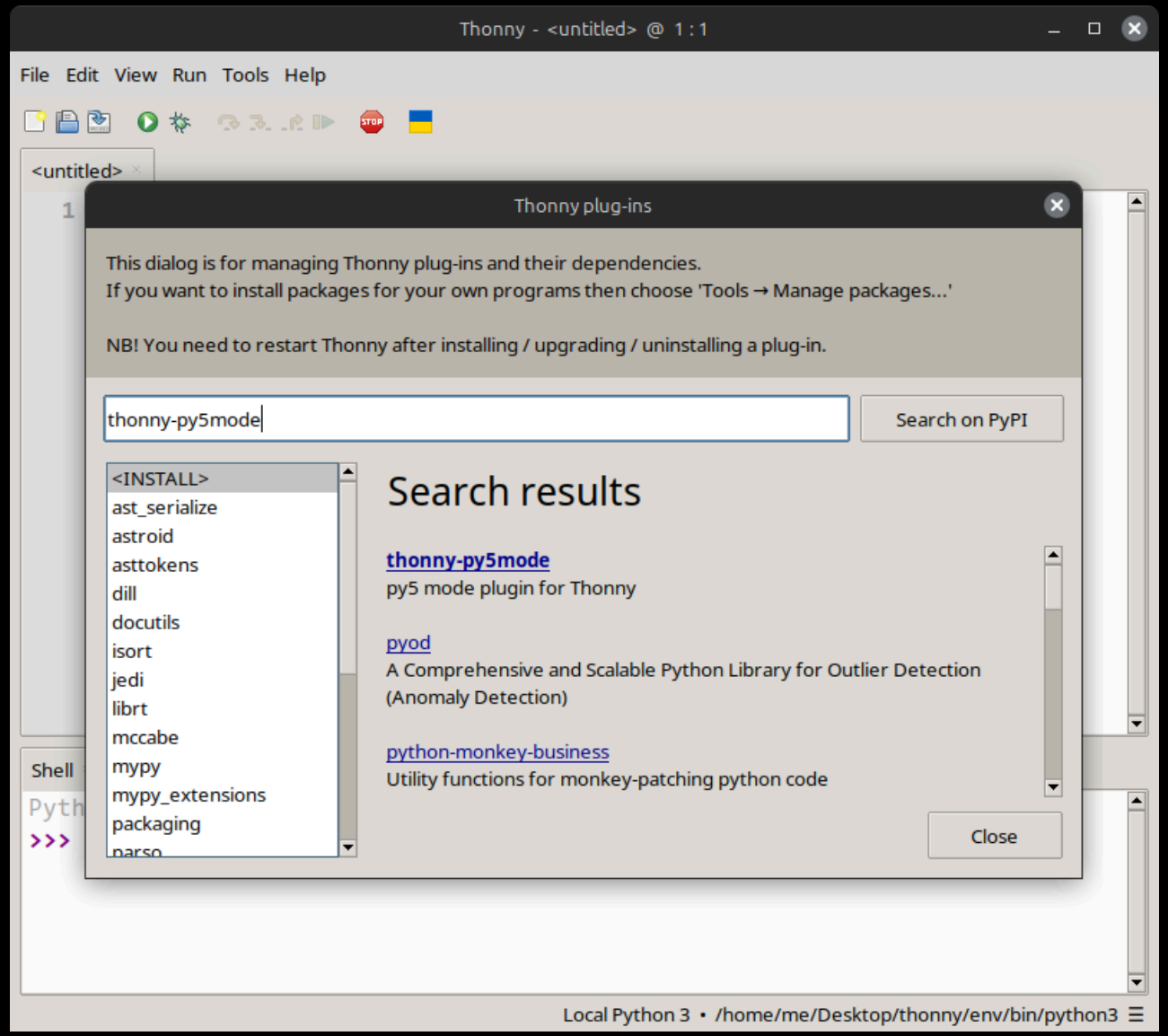
Process

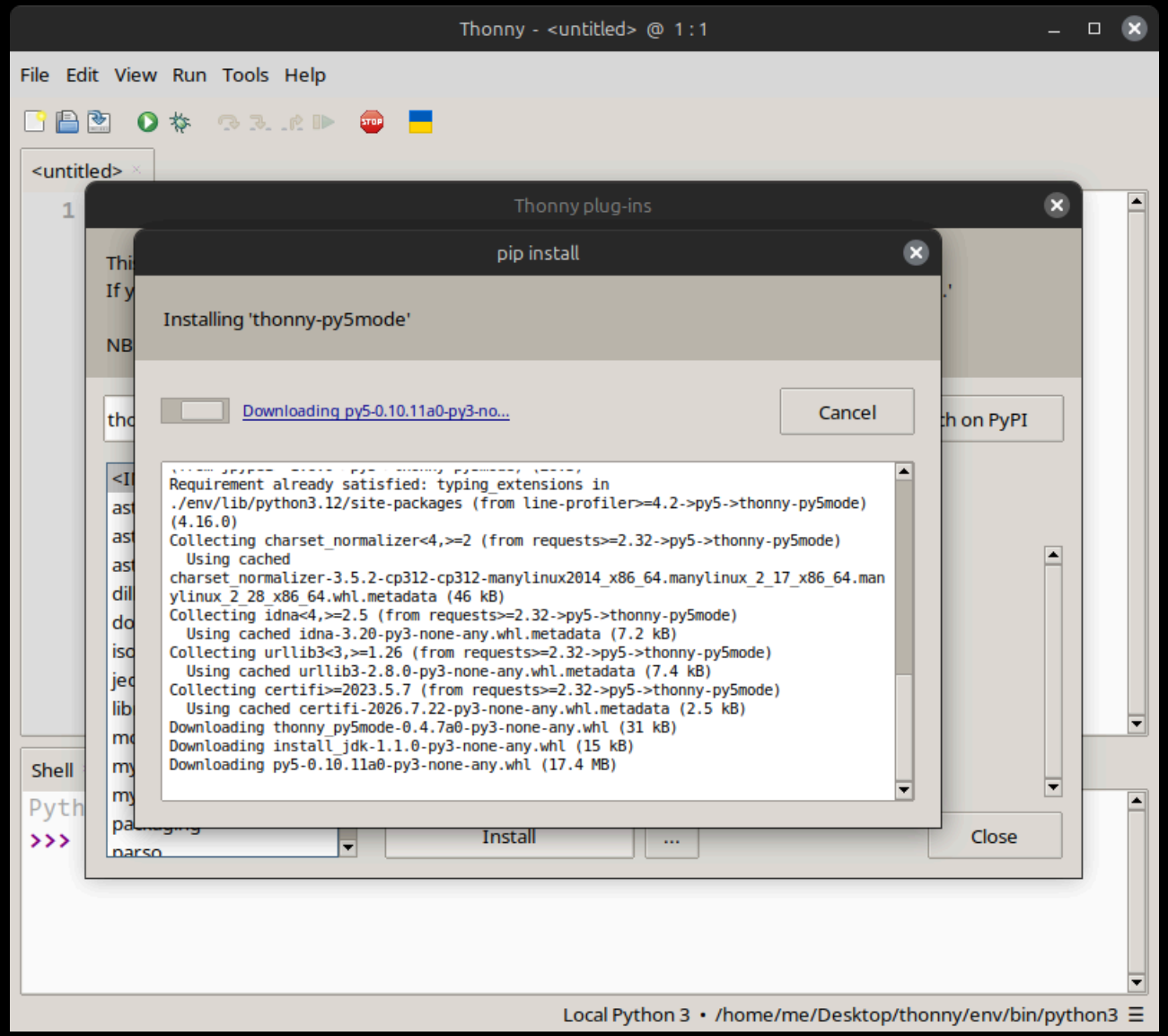
1. Install Thonny
2. Add plugin

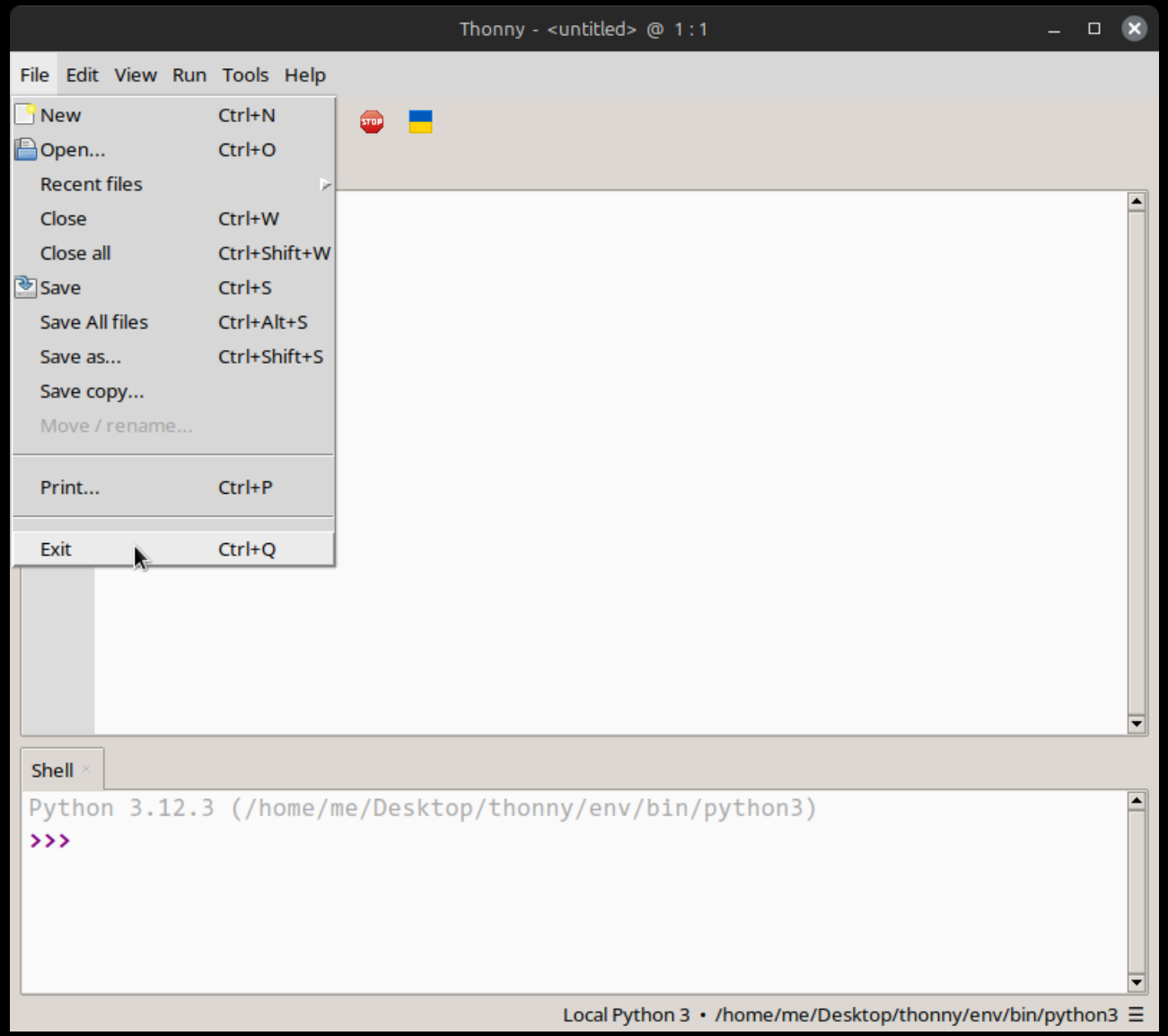

Language:
English (US) [default]
Initial settings:
Standard
Let's go!


Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help
<untitled>
1
Shell
Python 3.12.3 (/home/me/Desktop/thonny/env/bin/python3)
>>>
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

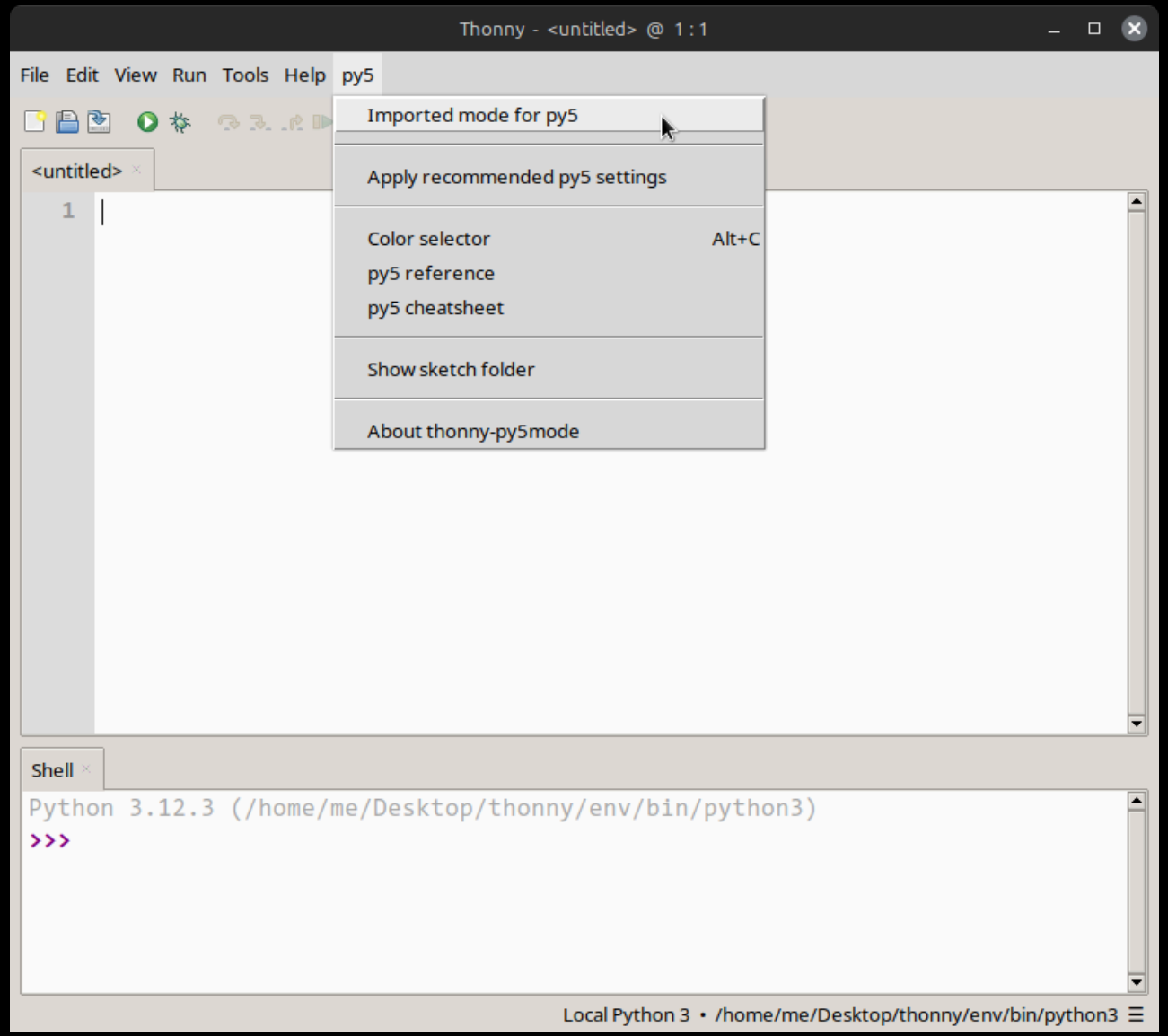

Thonny - <untitled> @ 1 : 1
File
Edit
View
Run
Tools
Help
Manage packages...
Open system shell...
Open Thonny program folder...
Open Thonny data folder...
Manage plug-ins...
Options...
<untitled>
1
Shell
Python 3.12.3 (/home/me/Desktop/thonny/env/bin/python3)
>>>
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

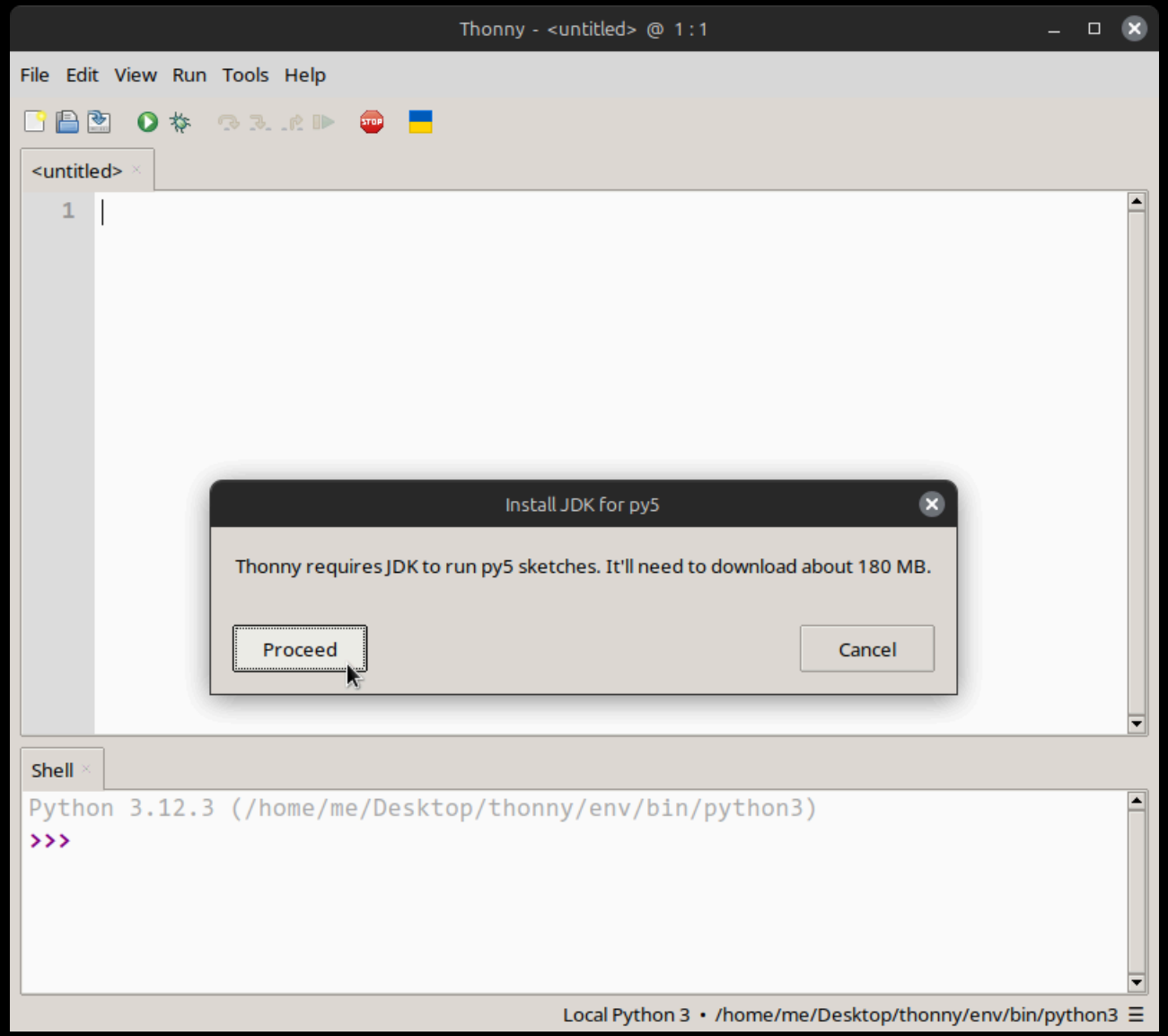

Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help
<untitled>
1
Thonny plug-ins
This dialog is for managing Thonny plug-ins and their dependencies.
If you want to install packages for your own programs then choose 'Tools → Manage packages...'
NB! You need to restart Thonny after installing / upgrading / uninstalling a plug-in.
thonny-py5mode
Search on PyPI
<INSTALL>
ast_serialize
astroid
asttokens
dill
docutils
isort
jedi
librt
mccabe
mypy
mypy_extensions
packaging
parso
Search results
thonny-py5mode
py5 mode plugin for Thonny
pyod
A Comprehensive and Scalable Python Library for Outlier Detection (Anomaly Detection)
python-monkey-business
Utility functions for monkey-patching python code
Close
Shell
>>>
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

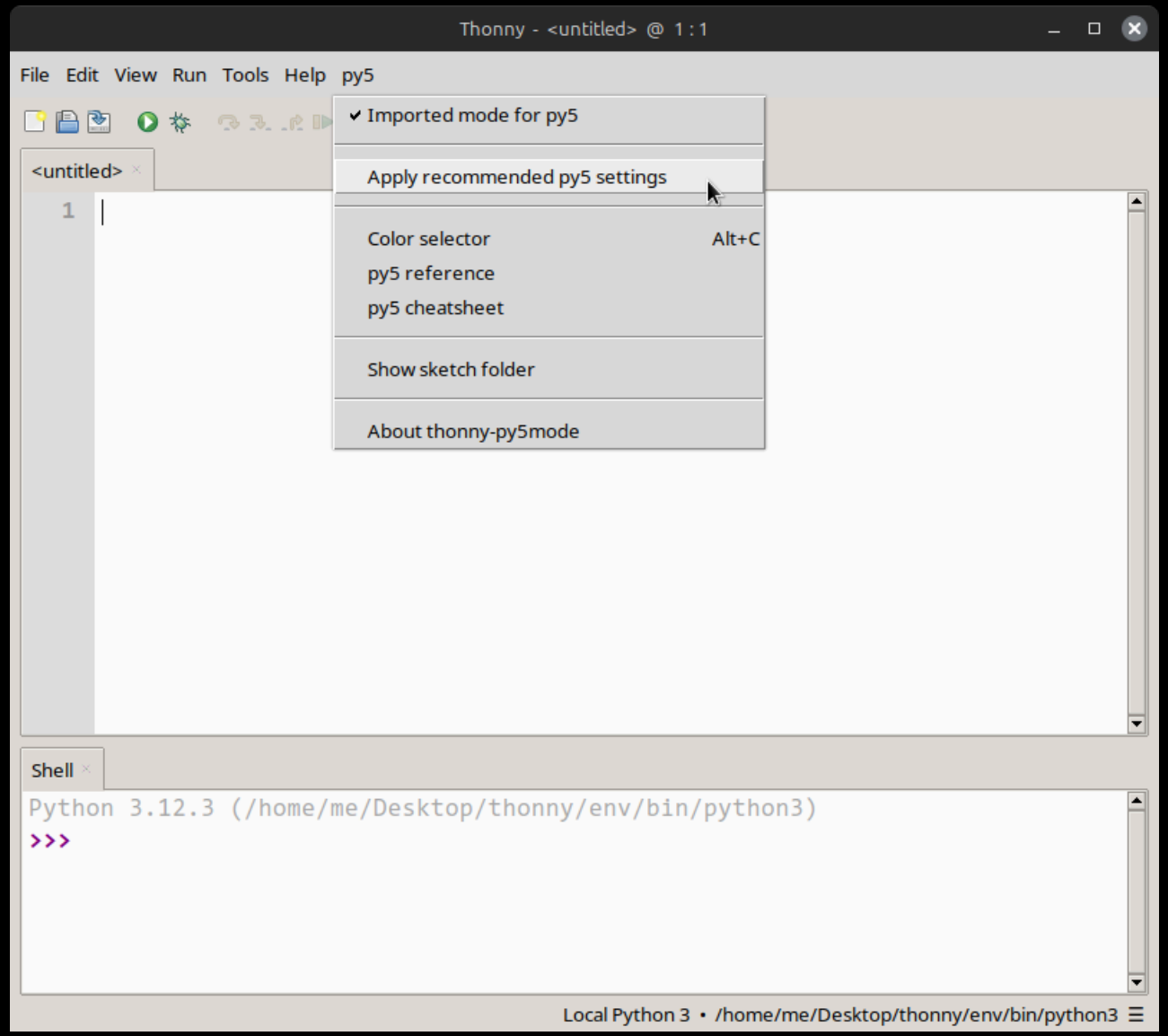

Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help
<untitled>
Thonny plug-ins
pip install
Installing 'thonny-py5mode'
Downloading py5-0.10.11a0-py3-no...
Cancel
Requirement already satisfied: typing_extensions in
./env/lib/python3.12/site-packages (from line-profiler>=4.2->py5->thonny-py5mode)
(4.16.0)
Collecting charset_normalizer<4,>=2 (from requests>=2.32->py5->thonny-py5mode)
Using cached
charset_normalizer-3.5.2-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.man
ylinux_2_28_x86_64.whl.metadata (46 kB)
Collecting idna<4,>=2.5 (from requests>=2.32->py5->thonny-py5mode)
Using cached idna-3.20-py3-none-any.whl.metadata (7.2 kB)
Collecting urllib3<3,>=1.26 (from requests>=2.32->py5->thonny-py5mode)
Using cached urllib3-2.8.0-py3-none-any.whl.metadata (7.4 kB)
Collecting certifi>=2023.5.7 (from requests>=2.32->py5->thonny-py5mode)
Using cached certifi-2026.7.22-py3-none-any.whl.metadata (2.5 kB)
Downloading thonny_py5mode-0.4.7a0-py3-none-any.whl (31 kB)
Downloading install_jdk-1.1.0-py3-none-any.whl (15 kB)
Downloading py5-0.10.11a0-py3-none-any.whl (17.4 MB)
Install
...
Close
Shell
>>>
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

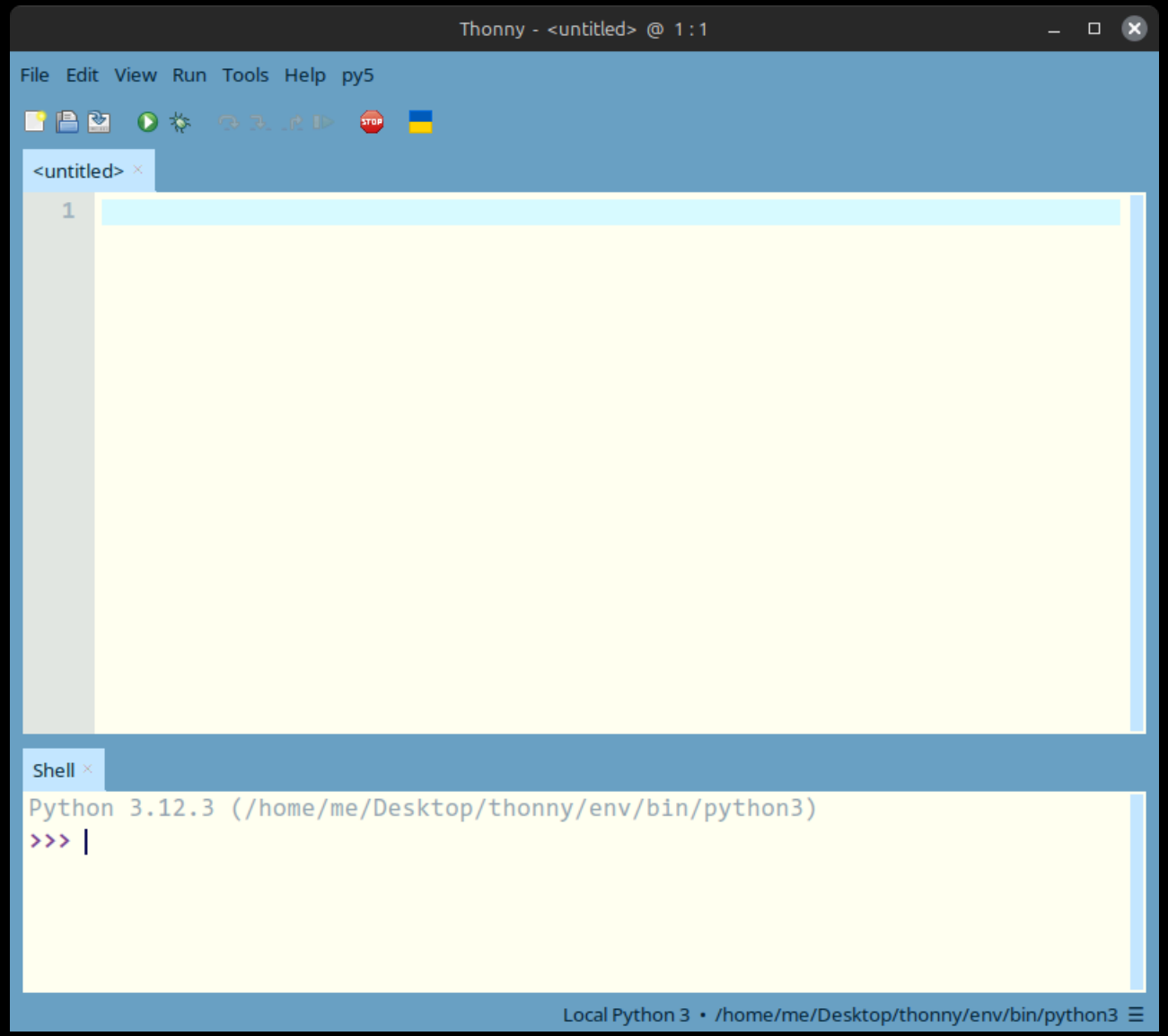

Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help
New Ctrl+N
Open... Ctrl+O
Recent files
Close Ctrl+W
Close all Ctrl+Shift+W
Save Ctrl+S
Save All files Ctrl+Alt+S
Save as... Ctrl+Shift+S
Save copy...
Move / rename...
Print... Ctrl+P
Exit Ctrl+Q
Shell
Python 3.12.3 (/home/me/Desktop/thonny/env/bin/python3)
>>>
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

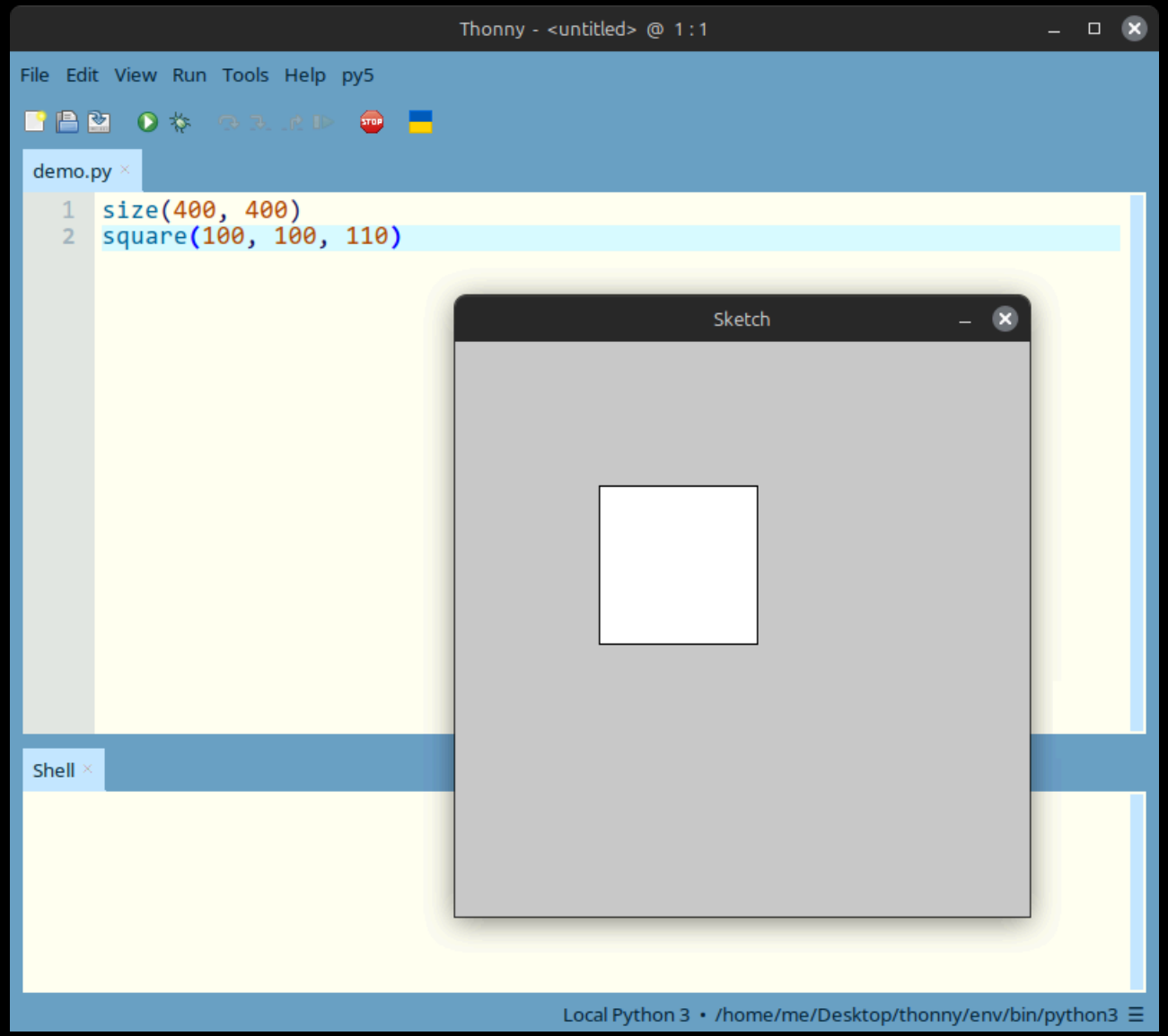
... restarting ...

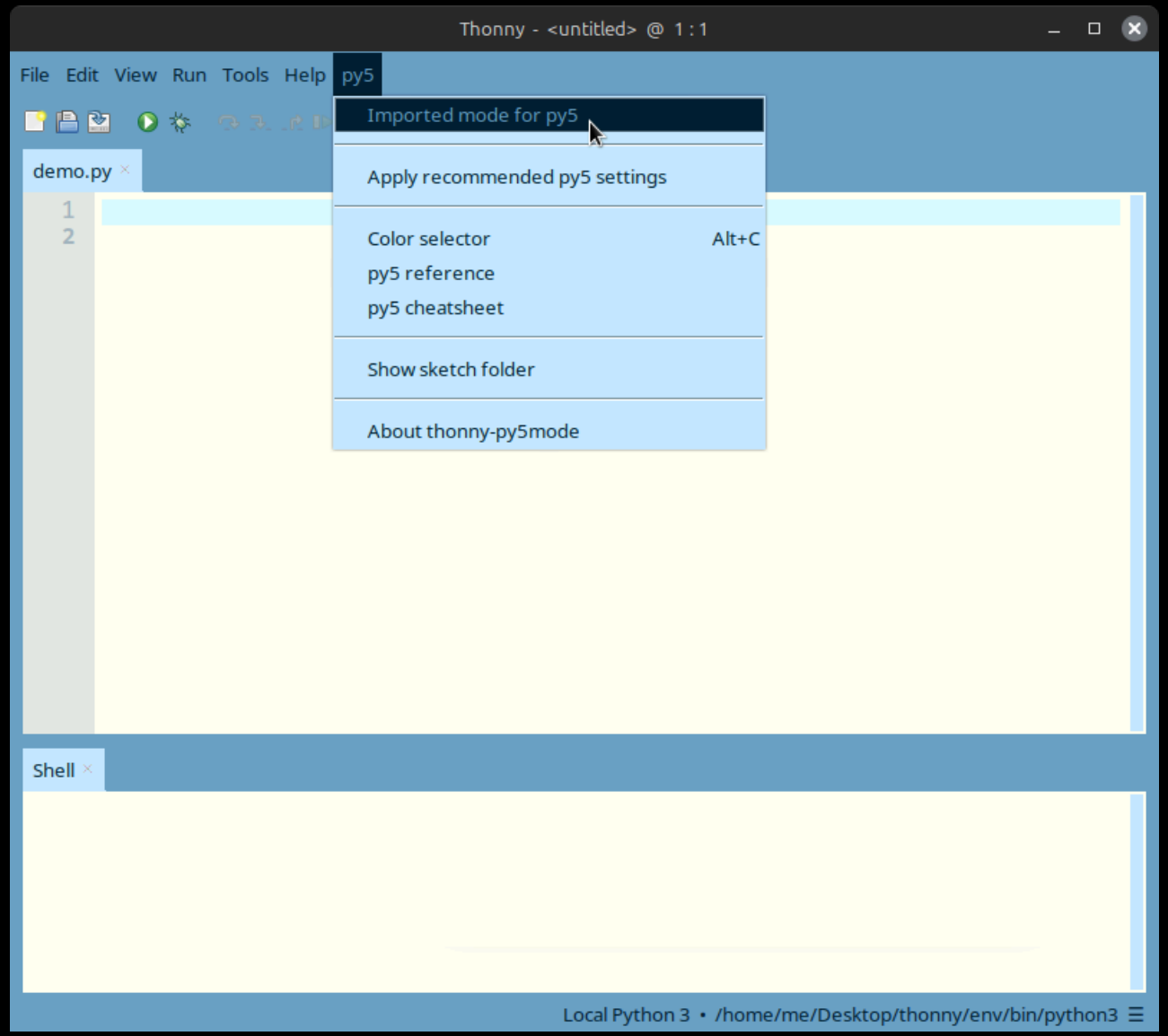

Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help py5
Imported mode for py5
Apply recommended py5 settings
Color selector Alt+C
py5 reference
py5 cheatsheet
Show sketch folder
About thonny-py5mode
<untitled>
1
Shell
Python 3.12.3 (/home/me/Desktop/thonny/env/bin/python3)
>>>
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

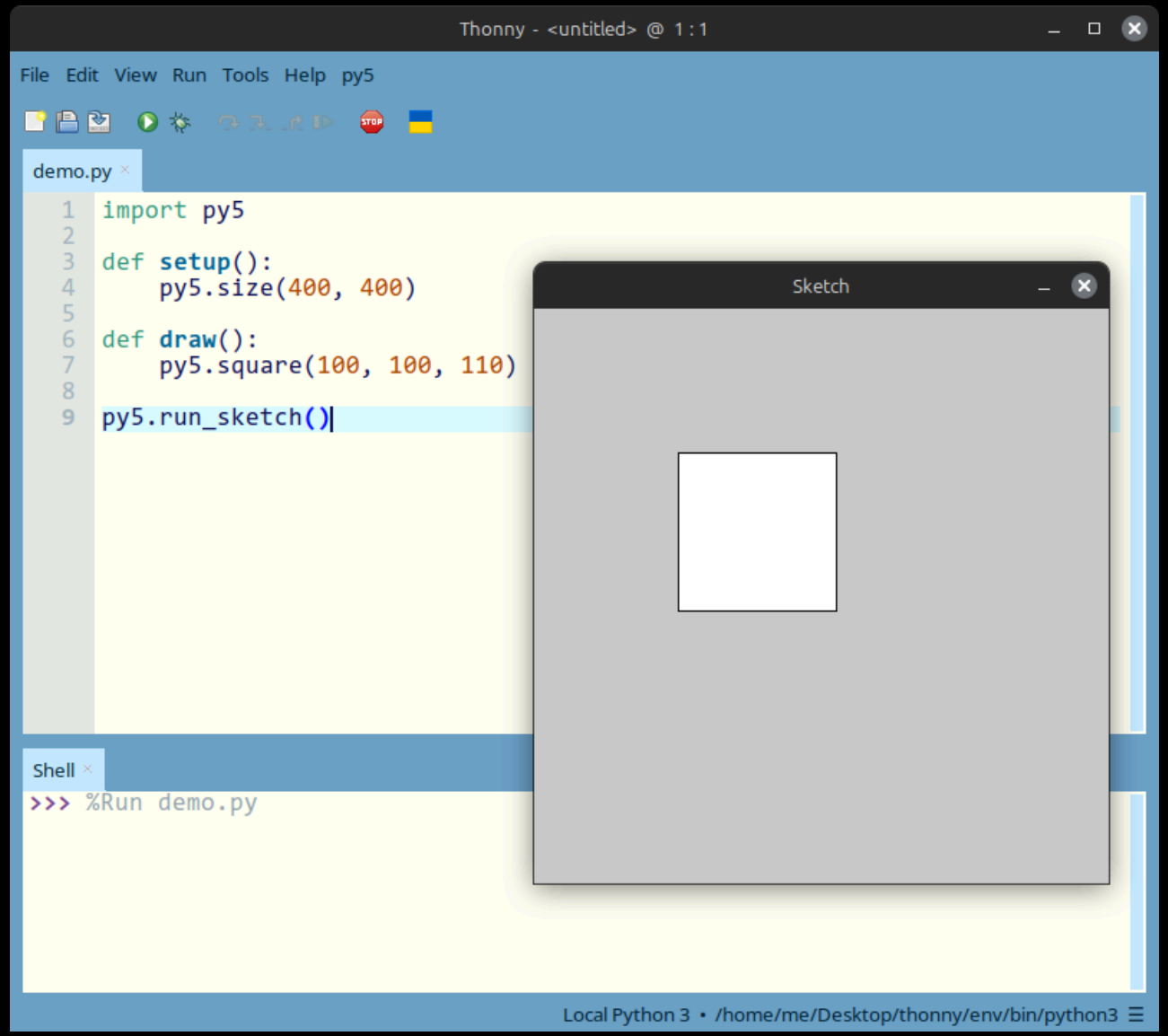

Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help
<untitled>
1
Install JDK for py5
Thonny requires JDK to run py5 sketches. It'll need to download about 180 MB.
Proceed
Cancel
Shell
Python 3.12.3 (/home/me/Desktop/thonny/env/bin/python3)
>>>
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3


Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help py5
Imported mode for py5
Apply recommended py5 settings
Color selector Alt+C
py5 reference
py5 cheatsheet
Show sketch folder
About thonny-py5mode
<untitled>
1
Shell
Python 3.12.3 (/home/me/Desktop/thonny/env/bin/python3)
>>>
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3


Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help py5
<untitled>
1
Shell
Python 3.12.3 (/home/me/Desktop/thonny/env/bin/python3)
>>>
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3


Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help py5
demo.py
1 size(400, 400)
2 square(100, 100, 110)
Sketch
Shell
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3


Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help py5
Imported mode for py5
Apply recommended py5 settings
Color selector Alt+C
py5 reference
py5 cheatsheet
Show sketch folder
About thonny-py5mode
demo.py
1
2
Shell
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3


Thonny - <untitled> @ 1 : 1
File Edit View Run Tools Help py5
demo.py
1 import py5
2
3 def setup():
4 py5.size(400, 400)
5
6 def draw():
7 py5.square(100, 100, 110)
8
9 py5.run_sketch()
Sketch
Shell
>>> %Run demo.py
Local Python 3 • /home/me/Desktop/thonny/env/bin/python3

py5

py5 » Simple Sketch

```
def setup():
    size(640, 240)

def draw():
    background(255)
    circle(mouse_x, mouse_y, 48)
```

py5 » Porting Approach

- **Translate Directly** -- vectors, forces, particles: near line-for-line
- **Rethink Logic** -- p5.js-exclusive libraries need substitutes
- **Simplify Code** -- Python idioms shorten the original JavaScript

py5 » Substitutions

p5.js (original)	Python (py5)
Matter.js	Pymunk (Chipmunk2D wrapper)
toxiclibs.js	Toxiclibs *Java* library via the JVM
ml5.js	scikit-learn / bespoke NumPy

Translate Directly

Translate Directly » py5 Naming

```
// p5.js
function setup() {
  createCanvas(640, 240);
}
function draw() {
  circle(mouseX, mouseY, 48);
}

# py5
def setup():
    size(640, 240)
def draw():
    circle(mouse_x, mouse_y, 48)
```

Translate Directly » Renames to Know

p5.js	py5	Why
`createCanvas()`	`size()`	Processing Java
`map()`	`remap()`	`map()` Python built-in
`mouseX`, `strokeWeight()`	`mouse_x`, `stroke_weight()`	snake_case
`createVector`	`Py5Vector2D`, `Py5Vector3D`	real class

Translate Directly » Randomness & Noise

```
// p5.js
let x = random(width);
let g = randomGaussian(0, 1);
let n = noise(xoff);

# py5
x = random(width)                # py5's own, not the random module
g = random_gaussian(0, 1)
n = noise(xoff)
```

Translate Directly » Vector Operations

```
// p5.js -- createVector() must be called inside setup()
position = createVector(100, 100);
velocity = createVector(2.5, 2);
position.add(velocity);

# py5 -- a real class, usable anywhere
position = Py5Vector2D(100, 100)
velocity = Py5Vector2D(2.5, 2)
position += velocity
```

Translate Directly » Vector Attributes

```
// p5.js
let mouse = createVector(mouseX, mouseY);
mouse.sub(center);
const m = mouse.mag();  // method call

# py5
mouse = Py5Vector2D(mouse_x, mouse_y)
mouse -= center
m = mouse.mag           # plain attribute
```

Translate Directly » Class

```
// p5.js
class Mover {
  constructor(x, y) {
    this.position = createVector(x, y);
  }
}

# py5
class Mover:
    def __init__(self, x, y):
        self.position = Py5Vector2D(x, y)
```

Translate Directly » Class Methods

```
def seek(self, target: Py5Vector2D) -> Py5Vector2D:
    # Calculate the desired velocity to target at max speed.
    desired = target - self.position
    if desired.mag == 0: return Py5Vector2D()
    desired.normalize()
    desired.set_mag(self.max_speed)
    # Reynolds' formula for steering force.
    steer = desired - self.velocity
    steer.set_limit(self.max_force)
    return steer
```

Translate Directly » Caveat:
PY5 IMPORTED MODE CODE

```
# mover.py
# PY5 IMPORTED MODE CODE

class Mover:
    def __init__(self):  # no width/height parameters needed
        self.position = Py5Vector2D(width / 2, height / 2)

    def check_edges(self):
        if self.position.x > width:  # always the live value
            self.position.x = 0
```

Rethink Logic

Rethink Logic » Scope Is Different

```
// p5.js -- outer variables just work
let position;
function draw() { position.add(velocity); }

# py5 -- assignment needs an explicit declaration
def draw():
    global position
    position += velocity
```

Rethink Logic » No preload()

```
// p5.js -- assets load asynchronously before setup()
let img;
function preload() {
  img = loadImage('texture.png');
}

# py5 -- loading is synchronous: just do it in setup()
def setup():
    global img
    size(640, 240)
    img = load_image('texture.png')
```

Rethink Logic » Deleting While Iterating

```
// p5.js -- loop backward to splice safely
for (let i = particles.length - 1; i >= 0; i--) {
  particles[i].run();
  if (particles[i].isDead()) particles.splice(i, 1);
}

# py5 -- same backward-loop technique works ...
for i in range(len(particles) - 1, -1, -1):
    particles[i].run()
    if particles[i].dead:
        del particles[i]
```

Rethink Logic » Physics: Matter.js » Pymunk

```
from pymunk import Body, moment_for_box, Poly, Space

engine = Space()                              # world/engine = Space
engine.gravity = (0, 1.0 * SCALE_GRAVITY) # calibration constant
```

Rethink Logic » Pymunk: Calibrating Two Engines

```
"""
Since the Matter.js and Pymunk engines use different units
and solvers, these values provide an *approximate* conversion
for matching visual behavior.
"""
SCALE_GRAVITY = 900.0
FPS = 60.0
SCALE_TIME = FPS
SCALE_VELOCITY = SCALE_TIME
SCALE_FRICTION = SCALE_TIME
```

Rethink Logic » Toxiclibs: Skip the Port, Use the Original

```
# The actual Java library, imported straight into Python
from toxi.geom import Vec2D, Rect
from toxi.physics2d import VerletPhysics2D, VerletSpring2D

physics = VerletPhysics2D()
physics.setWorldBounds(Rect(0, 0, width, height))
```

Rethink Logic » Wrapping Java Objects

```
class Particle:  # Python wrapper around a Java VerletParticle2D

    def __init__(self, x, y, r):
        self.p = VerletParticle2D(x, y)  # underlying Java object

# Must add everything to the world -- pass the Java object via .p
physics.addParticle(particle_1.p)
physics.addSpring(VerletSpring2D(particle_1.p, particle_2.p, length, strength))
```

Rethink Logic » Machine Learning: ml5.js » scikit-learn

```
from sklearn.neural_network import MLPClassifier

classifier = MLPClassifier(hidden_layer_sizes=(16,), max_iter=2500)
classifier.fit(inputs, outputs)   # trains before setup() runs
```

Rethink Logic » Neuroevolution: a Bespoke NumPy Brain

```
class Brain:
    """Vectorized NumPy brain with GA helpers."""

    def __init__(self, inputs, outputs, hidden=8):
        self.w_ih = _rng.uniform(-1.0, 1.0, (hidden, inputs))
        self.b_h  = _rng.uniform(-1.0, 1.0, (hidden,))
        # ... w_ho, b_o ...

    def forward(self, inputs):
        h = np.tanh(self.w_ih @ x + self.b_h)
        return (self.w_ho @ h + self.b_o).tolist()
```

Simplify Code

Simplify Code » Operator Overloading

```
// p5.js -- verbose static/method calls
let force = p5.Vector.sub(attractor, mover);
force.normalize();
force.mult(strength);
mover.acceleration.add(force);

# py5 -- reads like the maths
force = (attractor - mover).norm * strength
mover.acceleration += force
```

Simplify Code » Comprehensions

```
// p5.js
let training = [];
for (let i = 0; i < 2000; i++) {
  training.push([random(-width/2, width/2), random(-height/2, height/2), 1]);
}

# py5
training = [
  [random(-width / 2, width / 2), random(-height / 2, height / 2), 1]
  for _ in range(2000)
]
```

Simplify Code » Filtering Particles

```
// p5.js -- backward loop + splice
for (let i = particles.length - 1; i >= 0; i--) {
  if (particles[i].isDead()) particles.splice(i, 1);
}

# py5 -- rebuild the list declaratively
self.particles = [p for p in self.particles if not p.dead]
```

Simplify Code » Slices: Cellular Automata

```
// p5.js
let nextgen = cells.slice();

# py5
next_gen = cells[:]                  # copy a generation
cells = [0] * (width // W)           # build a row of dead cells
left, me, right = cells[i - 1], cells[i], cells[i + 1]
```

Simplify Code » Small Quality-of-Life Wins

- `position += velocity` -- augmented assignment on vectors
- `Particle(*self.origin)` -- vectors unpack as iterables
- `10_000` -- readable numeric literals
- Type hints: `def step(self) -> None:` documents intent
- No `let` / `const` / semicolons / braces
- Synchronous ML training -- no callback pyramids

Materials

Dedication
Acknowledgments
Introduction
0 Randomness
1 Vectors
2 Forces
3 Oscillation
4 Particle Systems
5 Autonomous Agents
6 Physics Libraries
7 Cellular Automata
8 Fractals
9 Evolutionary Computing
10 Neural Networks
11 Neuroevolution
Appendix: Creature Design
Additional Resources

Additional Resources

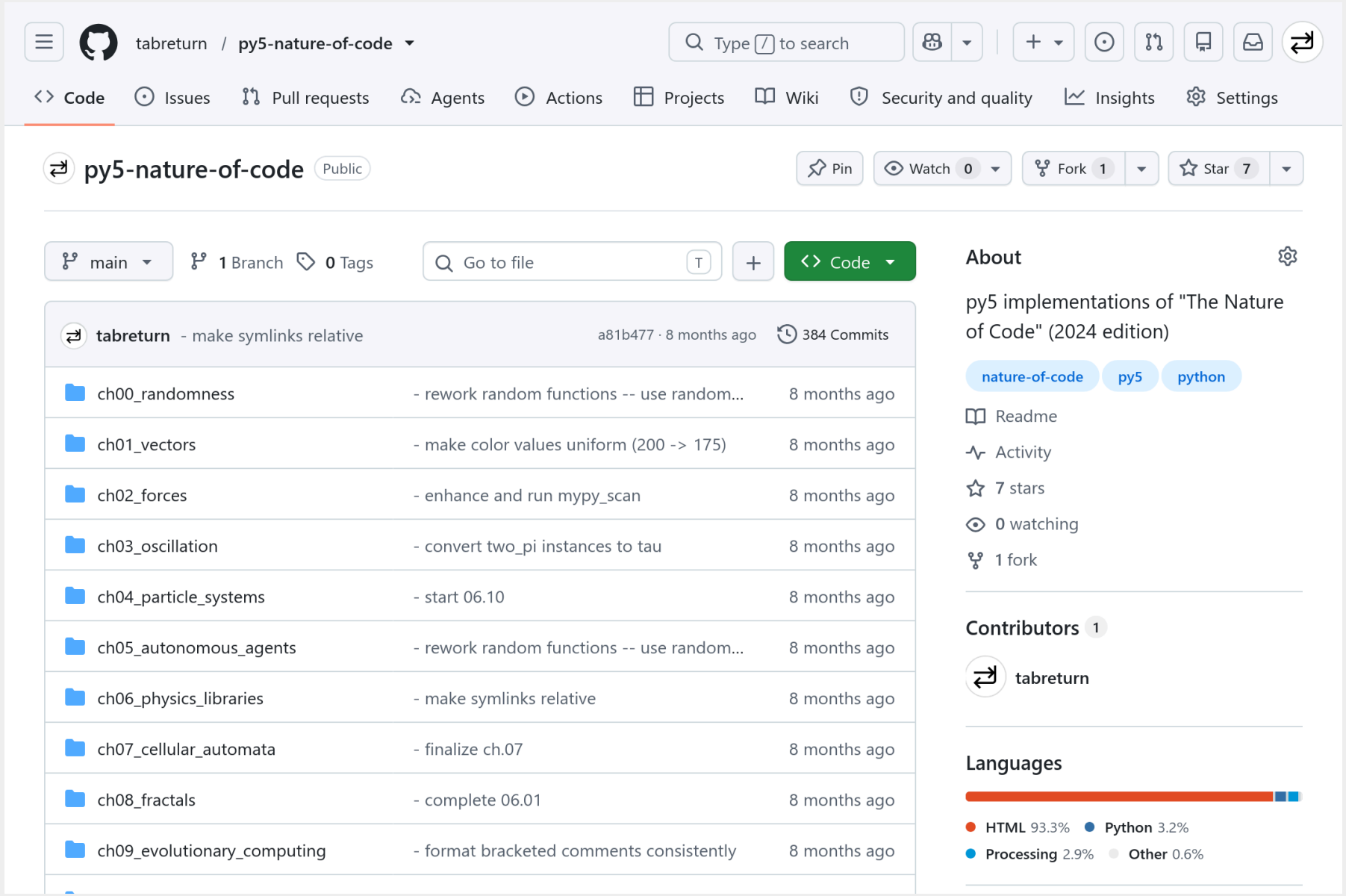
Ports of Code Examples to Other Languages

- Processing (Java) port by Coding Train community
- Python port with pygame by Niklas Hohn
- Another Python port with pygame by Oenarion
- py5 port (a version of Processing for Python) by tabreturn
- C++ port with openFrameworks by Jack B. Du
- C++/SDL by Alcides Schulz
- Go + Ebitengine by Jordan Philyaw
- If you are working on a port, please open an issue on GitHub with links and other relevant information!

Further Reading

Books

- Alexander, R. McNeill. *Principles of Animal Locomotion*. Princeton, NJ: Princeton University Press,



https://github.com/tabreturn/py5-nature-of-code

banner.png	- wip (11.02)	9 months ago
mypy_scan.py	- amend mypy_scan to handle getters/set...	8 months ago

README

Nature of Code (2), py5 Port

py5 implementations of Daniel Shiffman's "The Nature of Code" (2024 edition)

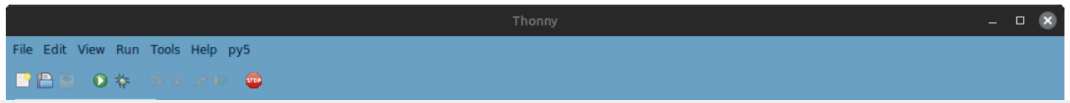
Built with py5 using Thonny-py5mode

I've taken a few liberties in my interpretation. First, the examples in the book and its accompanying p5.js sketches don't always align exactly. Second, in places, minor adjustments were necessary to support an idiomatic, py5-compatible Python implementation. Some chapters rely on p5.js-exclusive libraries, so I've had to employ substitutes/alternatives. Throughout, however, my goal has been to balance faithfulness to the original material with Python best practices and clarity for readers working through the book.

To run the sketches, install Thonny-py5mode and activate **py5 → Imported mode for py5** in the editor.

Find the original p5.js examples in the noc-book-2 repo, linked alongside each py5 sketch listed below, or in the Book's online Examples section.

Where text is displayed, the sketches use the *DejaVu Sans Mono* font.



Jekyll using Docker image
Package a Jekyll site using the jekyll/builder Docker image.
By GitHub Actions
Configure

Clojure
Build and test a Clojure project with Leiningen.
By GitHub Actions
Configure

More workflows

https://github.com/tabreturn/py5-nature-of-code

README

You'll see this marker in several modules, so that there's no need to add `from py5 import *` or `import py5`.

⚙️ Part 1: Inanimate Objects

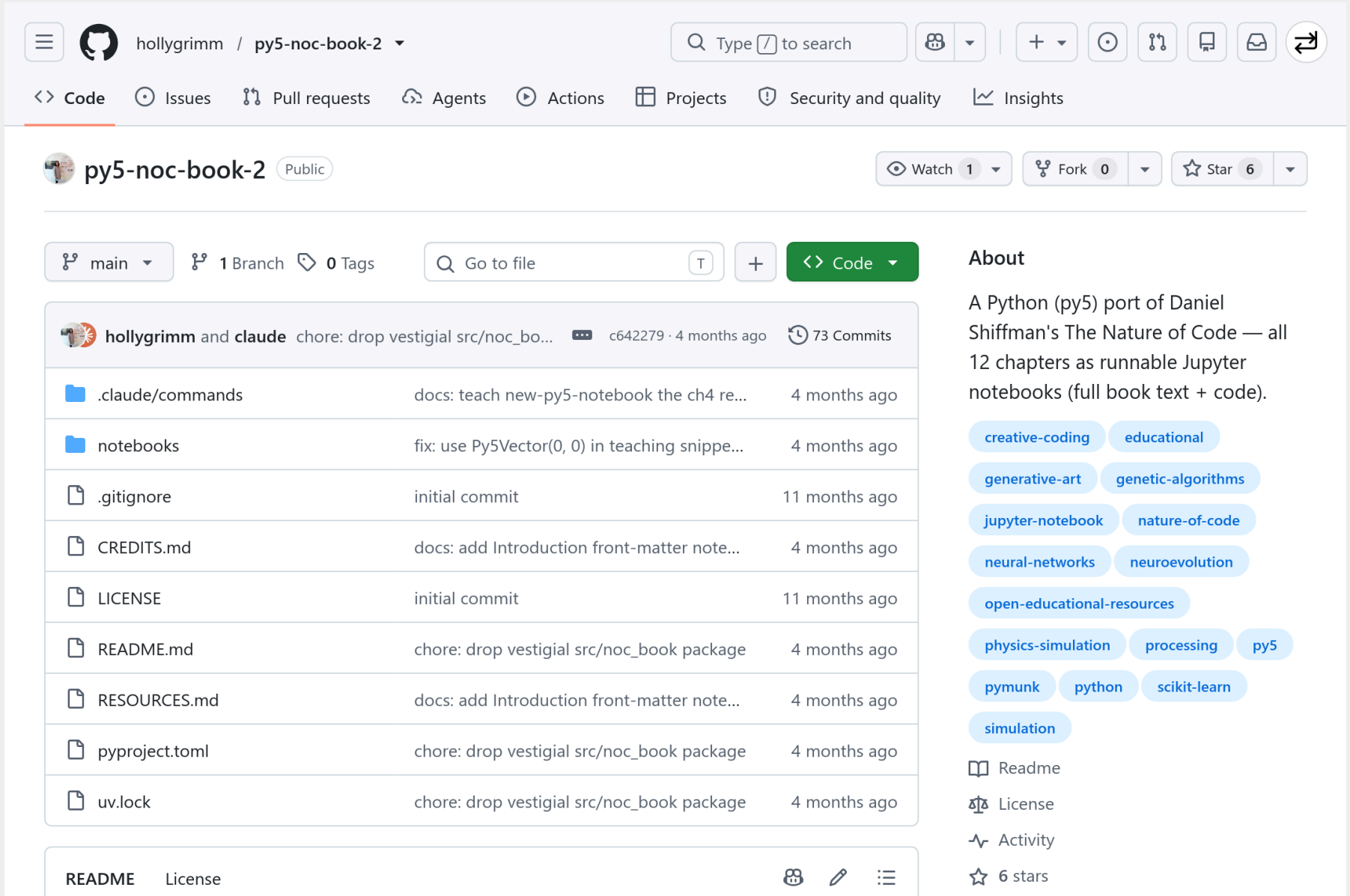
- ▸ Ch. 00 - Randomness
- ▸ Ch. 01 - Vectors
- ▸ Ch. 02 - Forces
- ▸ Ch. 03 - Oscillation
- ▸ Ch. 04 - Particle Systems

🧬 Part 2: It's Alive!

- ▸ Ch. 05 - Autonomous Agents
- ▸ Ch. 06 - Physics Libraries
- ▸ Ch. 07 - Cellular Automata
- ▸ Ch. 08 - Fractals

🧠 Part 3: Intelligence

https://github.com/tabreturn/py5-nature-of-code



https://github.com/hollygrimm/py5-noc-book-2

main py5-noc-book-2 / notebooks / 00_randomness.ipynb Top

Preview Code Blame 1916 lines (1916 loc) · 85.8 KB Raw

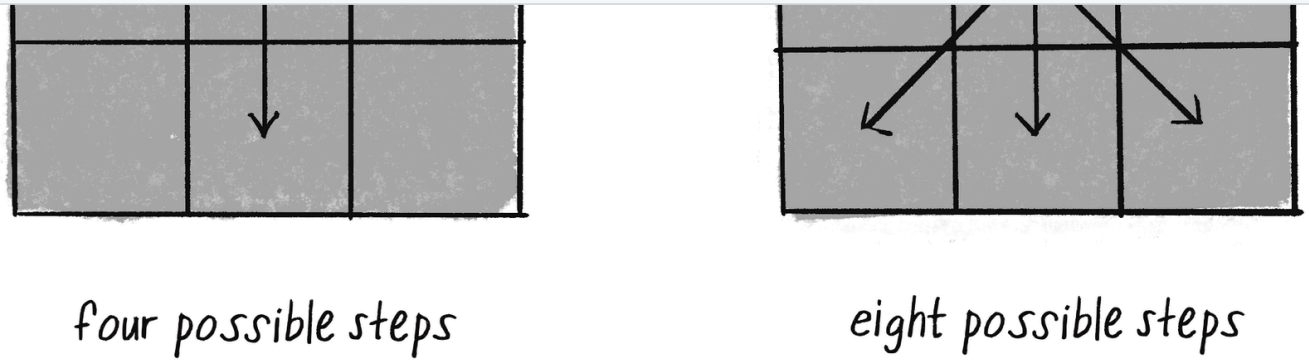


Figure 0.1: The steps of a random walker, with and without diagonals

To implement a `Walker` object that can step to any neighboring pixel (or stay put), I could pick a number from 0 to 8 (nine possible choices). However, another way to write the code would be to pick from three possible steps along the x-axis (–1, 0, or 1) and three possible steps along the y-axis:

In [ ]:

```
def step(self):
    # Yields –1, 0, or 1
    xstep = int(py5.random(3)) - 1
    ystep = int(py5.random(3)) - 1
    self.x += xstep
    self.y += ystep
```

Taking this further, I could get rid of `int()` and use the `random()` function's original floating-point numbers to create a continuous range of possible step lengths from –1 to 1, as shown next.

In [ ]:

```
def step(self):
    # Any floating-point number from –1 to 1
    xstep = py5.random(-1, 1)
```

https://github.com/hollygrimm/py5-noc-book-2

Takeaways

Takeaways

- **Most of the book translates directly** -- randomness, vectors, forces, oscillation, particles, steering, CA, fractals, and GA are near line-for-line
- **Renames to remember**: `map()` » `remap()`, camelCase » snake_case, `.mag()` » `.mag`
- **The `global` keyword** is the most common porting stumble
- **Library substitution**: Pymunk for Matter.js, original Java Toxiclibs via the JVM, scikit-learn / NumPy for ml5.js
- **Python often simplifies**: operator overloading, comprehensions, slices
- **AI drafts, you verify**: great at renaming passes, weak on library judgement and visual correctness

Try It Yourself

- Install **Thonny-py5mode**, activate *py5 » Imported mode*
- Port a chapter 1 or 2 sketch by hand -- it's mostly renaming
- Compare against the complete **py5-nature-of-code** port
- Read the book free online: natureofcode.com

End